

An aerial photograph of a tropical island with a central lagoon, surrounded by dense green vegetation and small white buildings. The water is a deep blue, and the island's coastline is irregular with small inlets and points.

Energy Reduction Strategies

A research workflow for improving software energy efficiency

Xutong Liu

UNIVERSITY
OF TWENTE.



Idle in the lobby for 30 seconds?
The game drops to 30 fps.

200 MWh

saved per day

≈ 14 wind turbines, all year

Epic Games, Reducing Fortnite's Power Consumption, 2023. Total across all players, for this and similar settings.

Fortnite found its saving in the lobby.

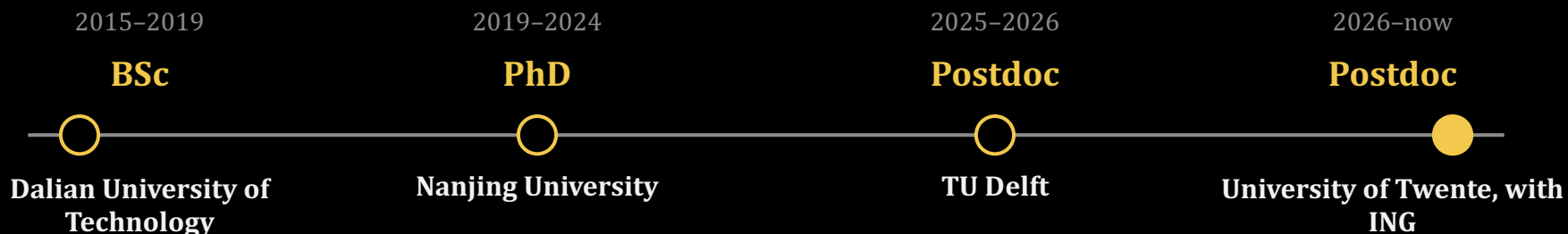
Where would you look in your own software?

So far: how to measure energy, and measure it right.

Today: how researchers go from a measurement to a greener system.

About me: green software and reliable LLMs

Xutong Liu · Postdoctoral researcher, University of Twente



Two research lines

Green software: measuring, profiling and reducing the energy of testing and CI

Reliable LLMs: confidence scores, reliability and stability of RAG-based LLM systems, with ING

Today's Intended Learning Outcomes

By the end of this lecture, you will be able to

- 6.1 Explain** an approach to save energy based on **software diversity**
- 6.2 Profile** a system to find its **energy bottlenecks**
- 7.1 Analyze** which techniques fit a **given context**
- 7.2 Apply** well-understood techniques to **improve energy efficiency**
- 7.3 Discuss open issues** in the scientific literature

Watch for six steps that recur in every case

ENERGY DEBUGGING

1 **Observe**

Why look here? What signal is worth chasing?

2 **Localize**

Where does the energy go, and why?

Watch for six steps that recur in every case

ENERGY DEBUGGING

- | | |
|-------------------|--|
| 1 Observe | Why look here? What signal is worth chasing? |
| 2 Localize | Where does the energy go, and why? |

OPTIMIZATION

- | | |
|----------------------------------|--|
| 3 Intervene | What do we change, and why that? |
| 4 Measure | Did it change energy? Compared to what? |
| 5 Evaluate | Is it really better, and does it still do its job? |
| 6 Reason about trade-offs | What did it cost, and where does it hold? |

From fine-grained to coarse-grained energy optimization

fine-grained

coarse-grained



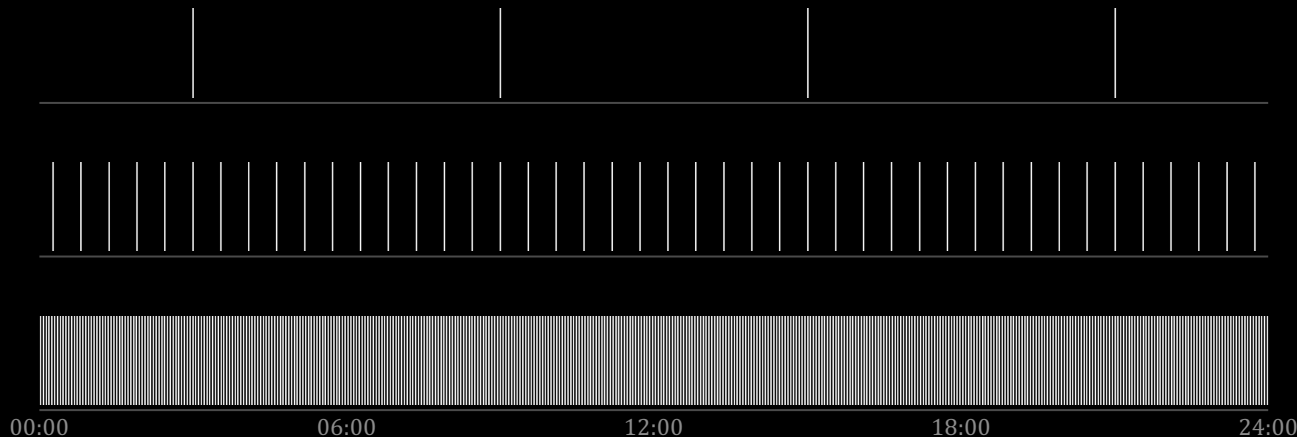
PyTorch tests every push to a pull request

How many times in one year?

A 1,600
about 4 a day

B 16,000
about 44 a day

C 160,000
about 440 a day



Each line is one start of the workflow, on an average day.

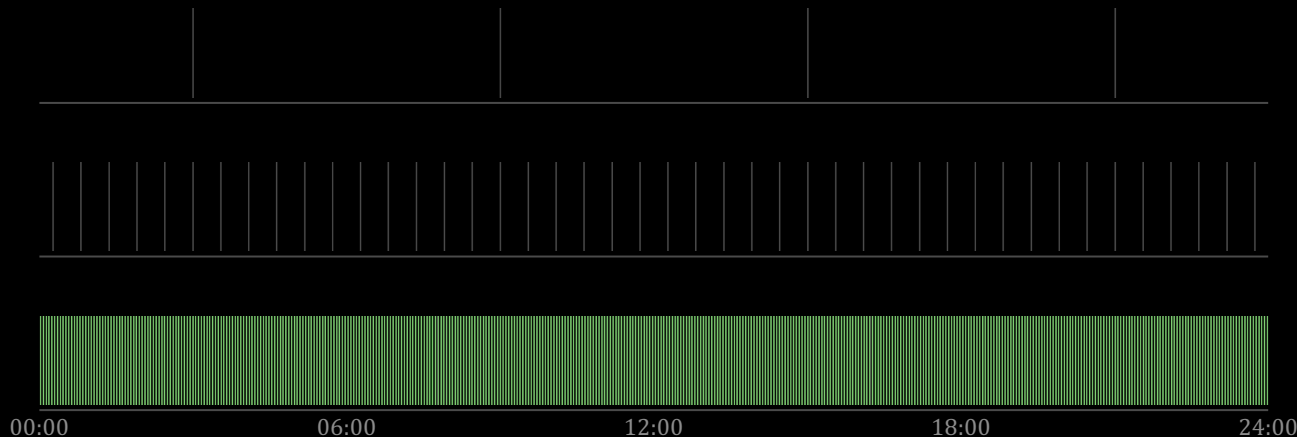
PyTorch tests every push to a pull request

Answer: **159,296 times: one start every three minutes, day and night**

A 1,600
about 4 a day

B 16,000
about 44 a day

C 160,000
about 440 a day



Each line is one start, on an average day. Each start can launch up to 115 test jobs.

Counted from the run numbers of PyTorch's "pull" workflow, 1 Oct 2025 to 30 Sep 2026. Job count from pull.yml, 5 Oct 2026.

Case 1 · Reducing the energy of slow tests

Motivation: slow tests cost energy on every CI run

$$\begin{array}{ccccccc} \text{1 slow test} & \times & \text{every push} & \times & \text{several} & = & \text{thousands of} \\ & & \text{and PR} & & \text{platforms} & & \text{runs} \\ \text{one minute, say} & & \text{CI reruns the suite} & & \text{OS and Python versions} & & \text{paid again and again} \end{array}$$

Case 1 · Reducing the energy of slow tests

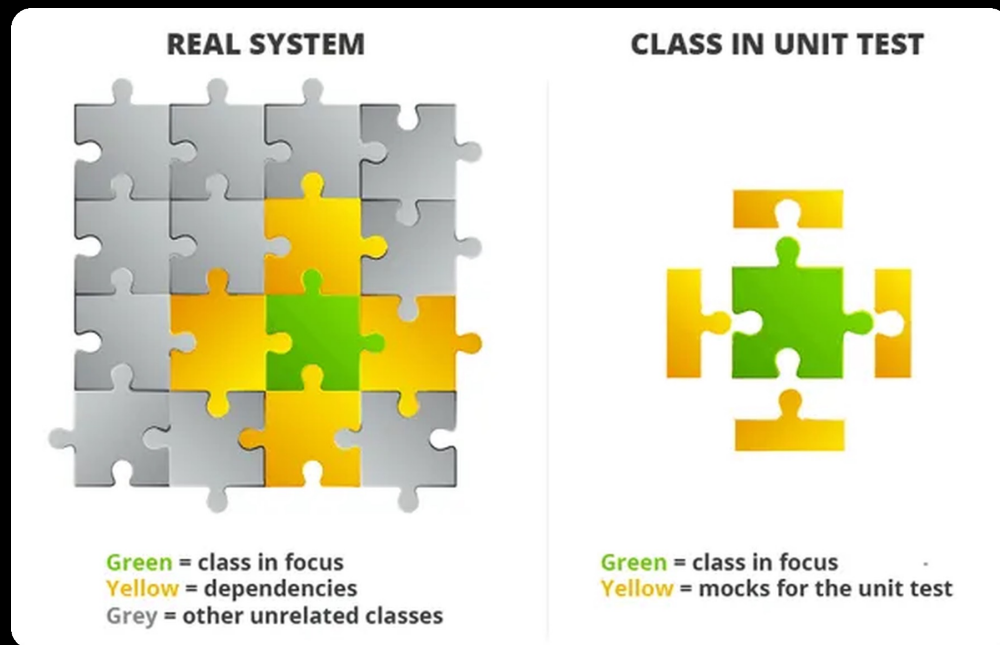
Motivation: slow tests cost energy on every CI run

$$\begin{array}{ccccccc} \text{1 slow test} & \times & \text{every push} & \times & \text{several} & = & \text{thousands of} \\ & & \text{and PR} & & \text{platforms} & & \text{runs} \\ \text{one minute, say} & & \text{CI reruns the suite} & & \text{OS and Python versions} & & \text{paid again and again} \end{array}$$

Developers already speed up slow tests, for fast feedback.

Could those speed-ups also save energy?

Mocking: replace a slow dependency with a stand-in



Real dependency

database, web service, files

the test waits: seconds

Mock

same shape, returns a prepared answer

no waiting: milliseconds

Only the dependency changes. The code under test and the assertions stay the same.

The gap: automated slow-test speed-ups for energy

Green testing

Optimizes the whole suite or CI pipeline, not the slow tests that cost energy

Mocking research

Mocking speeds up tests, but no one automates it for energy

LLM refactoring

Generic “make it greener” prompts give unstable gains

Verdecchia et al. 2025 · Spadini et al. 2017, 2019; Zhu et al. 2021 · Stivala et al. 2025

The gap: automated slow-test speed-ups for energy

Green testing

Optimizes the whole suite or CI pipeline, not the slow tests that cost energy

→ Find the slow tests that really cost energy

Mocking research

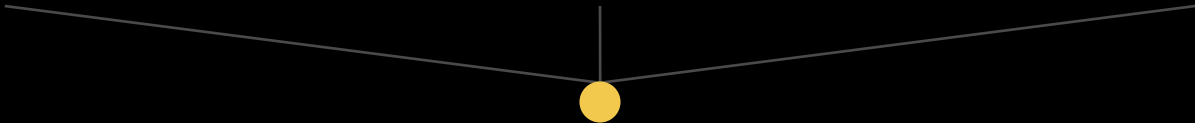
Mocking speeds up tests, but no one automates it for energy

→ Learn developers' fixes and automate the top one

LLM refactoring

Generic “make it greener” prompts give unstable gains

→ Give the agent one narrow task, and check the tests still work



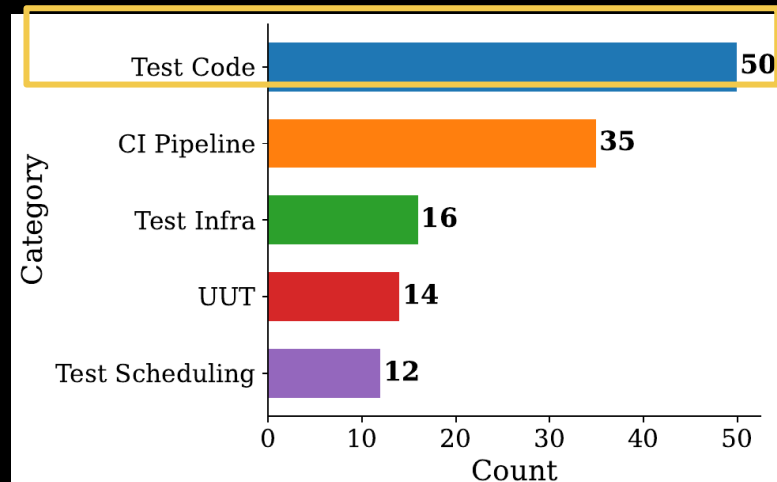
This work: find the real bottlenecks, automate what developers do, check the cost

Verdecchia et al. 2025 · Spadini et al. 2017, 2019; Zhu et al. 2021 · Stivala et al. 2025

RQ1 · How do developers fix slow tests?

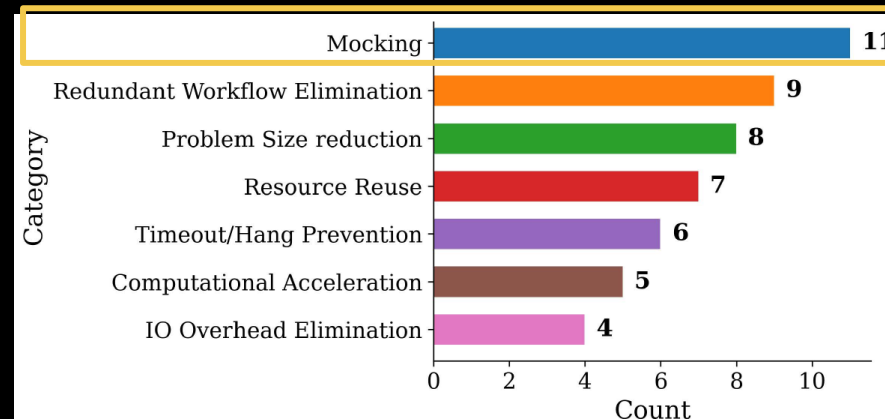
Answer: **Mostly in the test code, and mocking is the most common fix**

Which layer do they change?



Liu & Zaidman, 2026, Fig. 2 and 3 · 127 slow-test fixes from 108 Python projects

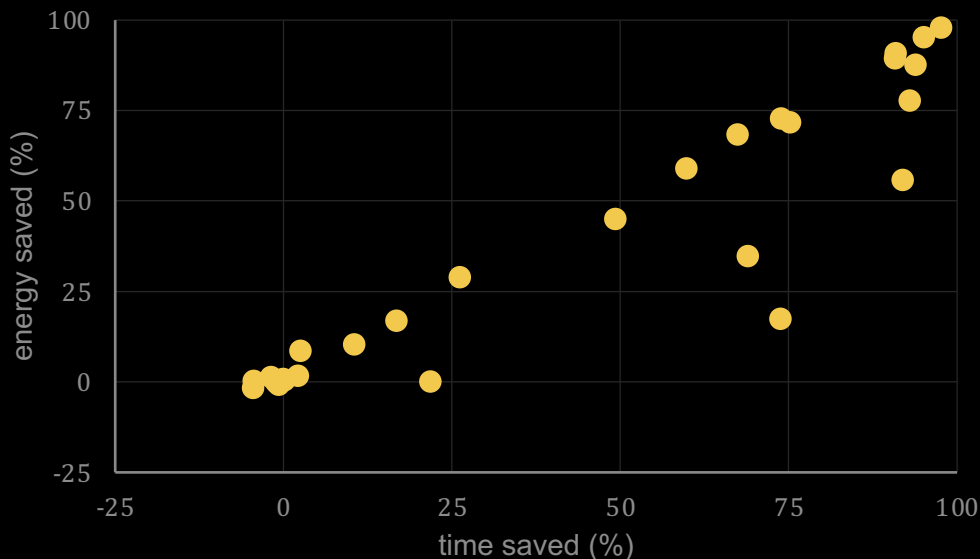
Inside test code: which mechanism?



Take intervention ideas from practice.

RQ2 · Does saving test time also save energy?

Answer: **Yes, strongly ($r = 0.929$), for these tests on this machine**



0.929

Spearman correlation, 27 developer fixes

before vs. after each fix, 30 runs each, one machine

Caveat: parallel code can finish faster and still use more energy.

Check a proxy before you trust it. Here: so test time can point to the energy bottlenecks

RQ3 · Can an AI agent cut test energy by mocking?

Setup: **only the slowest tests; the agent may only mock external dependencies**

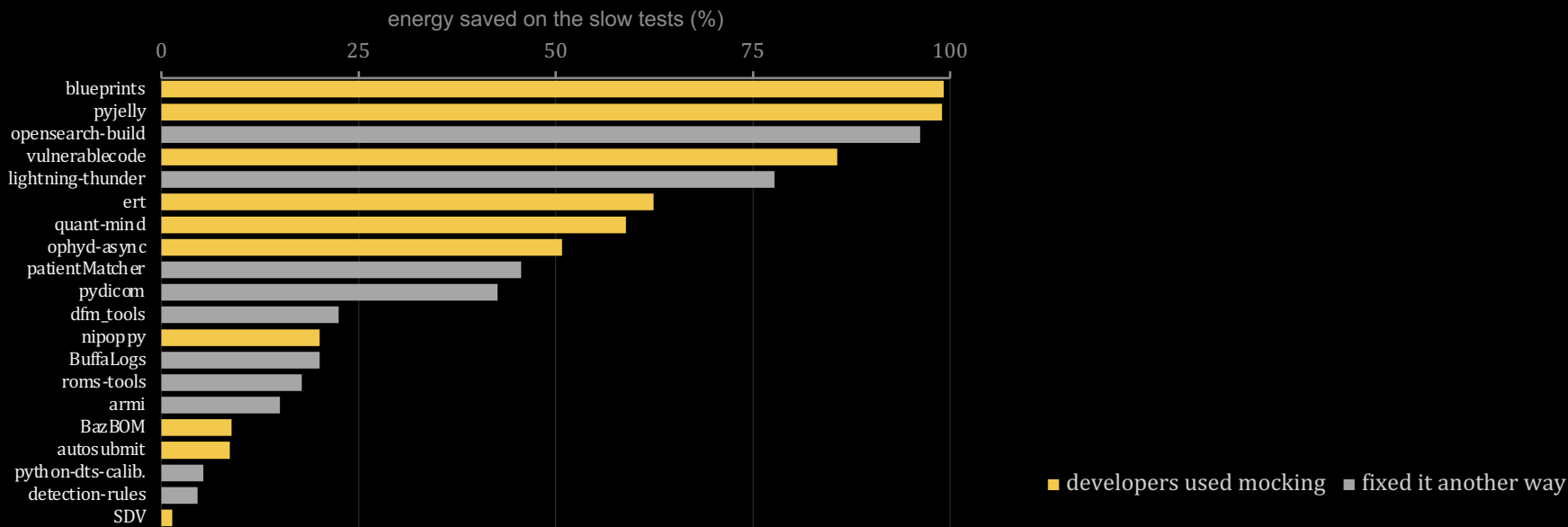
```
[INST] You are optimizing slow tests via mocking. Mock ONLY external dependencies to
      reduce test runtime without harming test effectiveness.
DO mock (external only): network / HTTP, database, file I/O, subprocess, and other time-
      consuming external interactions
DO NOT mock (internal):
(1) code under test / core computation
(2) verification logic (assertions, validation checks)
Prohibited
- changing timeouts/parameters/iterations or other speedup tricks that are not mocking
```

Liu & Zaidman, 2026, Listing 1 (top)

Green: what to mock **Red:** what not to touch

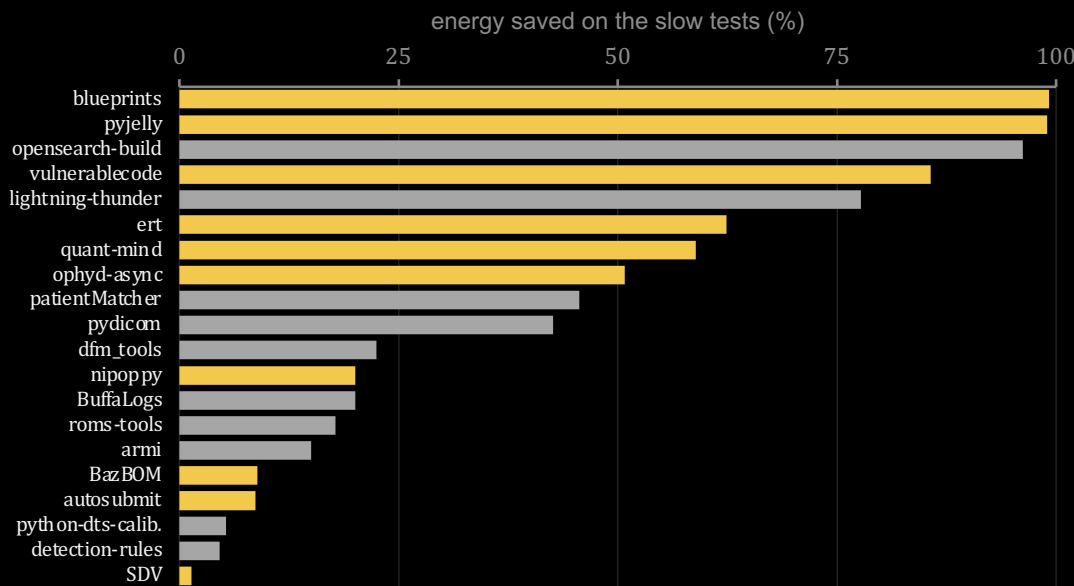
Software diversity (ILO 6.1): the real dependency and the mock are two implementations of one interface.

RQ3 · Can an AI agent cut test energy by mocking?



RQ3 · Can an AI agent cut test energy by mocking?

Answer: **Yes: significant cuts in time and energy in almost every project**



384 J

saved per run on average

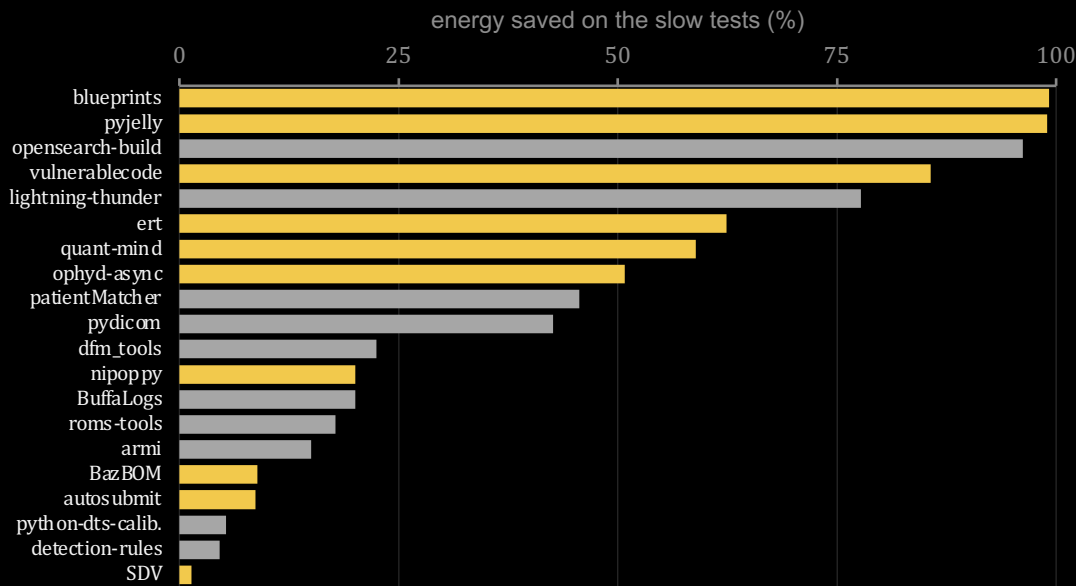
42%

less energy on average

■ developers used mocking ■ fixed it another way

RQ3 · Can an AI agent cut test energy by mocking?

Answer: **Yes: significant cuts in time and energy in almost every project**



384 J

saved per run on average

42%

less energy on average

Median: 40 J, 33%

Effect size (Cohen's d): large in 19 of 20 projects

■ developers used mocking ■ fixed it another way

Compare before vs. after, repeated, with statistics.

A joule is tiny; a year of CI runs is not



↑
40 cm

1 J

≈ lifting a glass of water to your mouth

A joule is tiny; a year of CI runs is not



↑
40 cm

1 J

≈ lifting a glass of water to your mouth

vulnerablecode, after mocking

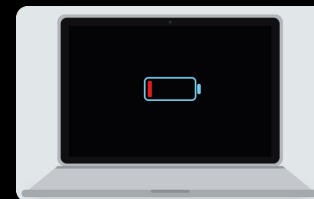
2,450 J saved per run

× 10 CI runs a day, for a year

≈ 9 MJ (2.5 kWh)

boil **27 litres** of water

empty a laptop battery **50 times**



Assumes 10 CI runs a day; CPU energy only. Kettle figure from the energy lecture (1 l of water: 330 kJ); laptop battery: 50 Wh = 180 kJ.

RQ4 · Do the mocked tests still catch bugs?

What could go wrong when we replace real dependencies with mocks?

RQ4 · Do the mocked tests still catch bugs?

Project	ΔE (J)	Cov_b (%)	Cov_a (%)	ΔCov (%)	Mut_b (%)	Mut_a (%)	ΔMut (%)
<i>Mockable Projects</i>							
autosubmit	15.2	10.8	10.8	-0.0	8.86	11.39	+2.53
BazBOM	2.10	3.4	3.0	-0.4	6.62	6.16	-0.46
blueprints	1320	25.1	25.1	0.0	98.93	11.50	-87.43
ert	264	25.1	25.1	0.0	99.09	99.05	-0.04
nipoppy	13.7	29.1	29.2	0.1	48.09	29.14	-18.96
ophyd-async	19.4	41.6	40.7	-0.9	53.03	22.68	-30.35
pyjelly	286	25.2	25.2	0.0	38.96	38.96	0.00
quant-mind	44.2	23.0	22.9	-0.1	34.54	34.54	0.00
SDV	1.57	15.2	15.1	-0.1	35.68	35.68	0.00
vulnerablecode	2450	11.0	11.0	0.0	40.43	37.83	-2.61
<i>Unknown Projects</i>							
armi	4.65	19.6	19.6	0.0	0.00	0.00	0.00
BuffaLogs	43.86	56.6	57.2	0.6	96.37	97.13	+0.76
detection-rules	36.5	40.8	38.8	-2.0	13.14	13.14	0.00
dfm_tools	12.7	12.8	11.0	-1.8	54.95	11.39	-43.56
lightning-thunder	1220	36.2	36.3	0.0	54.63	68.69	+14.06
opensearch-build	231	58.6	58.6	0.0	49.41	51.18	+1.76
patientMatcher	49.7	55.0	54.2	-0.8	81.89	44.11	-37.78
pydicom	8.57	25.7	21.3	-4.5	83.23	28.89	-54.34
python-dts-calibration	14.0	17.4	16.8	-0.6	6.12	28.22	+22.10
roms-tools	1650	29.9	28.4	-1.5	0.00	5.08	+5.08

Cov

% of code lines the tests execute

Mut

% of injected bugs the tests catch

RQ4 · Do the mocked tests still catch bugs?

Project	ΔE (J)	Cov_b (%)	Cov_a (%)	ΔCov (%)	Mut_b (%)	Mut_a (%)	ΔMut (%)
<i>Mockable Projects</i>							
autosubmit	15.2	10.8	10.8	-0.0	8.86	11.39	+2.53
BazBOM	2.10	3.4	3.0	-0.4	6.62	6.16	-0.46
blueprints	1320	25.1	25.1	0.0	98.93	11.50	-87.43
ert	264	25.1	25.1	0.0	99.09	99.05	-0.04
nipoppy	13.7	29.1	29.2	0.1	48.09	29.14	-18.96
ophyd-async	19.4	41.6	40.7	-0.9	53.03	22.68	-30.35
pyjelly	286	25.2	25.2	0.0	38.96	38.96	0.00
quant-mind	44.2	23.0	22.9	-0.1	34.54	34.54	0.00
SDV	1.57	15.2	15.1	-0.1	35.68	35.68	0.00
vulnerablecode	2450	11.0	11.0	0.0	40.43	37.83	-2.61
<i>Unknown Projects</i>							
armi	4.65	19.6	19.6	0.0	0.00	0.00	0.00
BuffaLogs	43.86	56.6	57.2	0.6	96.37	97.13	+0.76
detection-rules	36.5	40.8	38.8	-2.0	13.14	13.14	0.00
dfm_tools	12.7	12.8	11.0	-1.8	54.95	11.39	-43.56
lightning-thunder	1220	36.2	36.3	0.0	54.63	68.69	+14.06
opensearch-build	231	58.6	58.6	0.0	49.41	51.18	+1.76
patientMatcher	49.7	55.0	54.2	-0.8	81.89	44.11	-37.78
pydicom	8.57	25.7	21.3	-4.5	83.23	28.89	-54.34
python-dts-calibration	14.0	17.4	16.8	-0.6	6.12	28.22	+22.10
roms-tools	1650	29.9	28.4	-1.5	0.00	5.08	+5.08

Cov % of code lines the tests execute

Mut % of injected bugs the tests catch

Coverage **barely moves** (at most -4.5 points)

Mutation score **drops > 10 points** in 6 of 20 projects

RQ4 · Do the mocked tests still catch bugs?

Answer: **Mostly yes, but mocking internal code makes tests miss bugs**

Project	ΔE (J)	Cov _b (%)	Cov _a (%)	Δ Cov (%)	Mut _b (%)	Mut _a (%)	Δ Mut (%)
Mockable Projects							
autosubmit	15.2	10.8	10.8	-0.0	8.86	11.39	+2.53
BazBOM	2.10	3.4	3.0	-0.4	6.62	6.16	-0.46
blueprints	1320	25.1	25.1	0.0	98.93	11.50	-87.43
ert	264	25.1	25.1	0.0	99.09	99.05	-0.04
nipoppy	13.7	29.1	29.2	0.1	48.09	29.14	-18.96
ophyd-async	19.4	41.6	40.7	-0.9	53.03	22.68	-30.35
pyjelly	286	25.2	25.2	0.0	38.96	38.96	0.00
quant-mind	44.2	23.0	22.9	-0.1	34.54	34.54	0.00
SDV	1.57	15.2	15.1	-0.1	35.68	35.68	0.00
vulnerablecode	2450	11.0	11.0	0.0	40.43	37.83	-2.61
Unknown Projects							
armi	4.65	19.6	19.6	0.0	0.00	0.00	0.00
BuffaLogs	43.86	56.6	57.2	0.6	96.37	97.13	+0.76
detection-rules	36.5	40.8	38.8	-2.0	13.14	13.14	0.00
dfm_tools	12.7	12.8	11.0	-1.8	54.95	11.39	-43.56
lightning-thunder	1220	36.2	36.3	0.0	54.63	68.69	+14.06
opensearch-build	231	58.6	58.6	0.0	49.41	51.18	+1.76
patientMatcher	49.7	55.0	54.2	-0.8	81.89	44.11	-37.78
pydicom	8.57	25.7	21.3	-4.5	83.23	28.89	-54.34
python-dts-calibration	14.0	17.4	16.8	-0.6	6.12	28.22	+22.10
roms-tools	1650	29.9	28.4	-1.5	0.00	5.08	+5.08

Cov % of code lines the tests execute

Mut % of injected bugs the tests catch

Coverage **barely moves** (at most -4.5 points)

Mutation score **drops > 10 points** in 6 of 20 projects

There is almost always a trade-off. Here: energy vs. test effectiveness

Case 1 follows all six steps

Observe

Localize

Intervene

Measure

Evaluate

Trade-offs

Xutong Liu, Andy Zaidman. From Slow to Green: Agentic AI-Powered Mock Refactoring for Python Tests. Preprint, 2026.

Case 1 follows all six steps

Observe	a cost that repeats: tests run on every commit
Localize	pick the slowest tests, with time as a checked proxy
Intervene	mocking, the top fix mined from developers, kept narrow
Measure	before vs. after, 30 runs, statistics
Evaluate	mutation score, code coverage: do tests still catch bugs?
Trade-offs	energy vs. test effectiveness; only external slowness

Xutong Liu, Andy Zaidman. From Slow to Green: Agentic AI-Powered Mock Refactoring for Python Tests. Preprint, 2026.

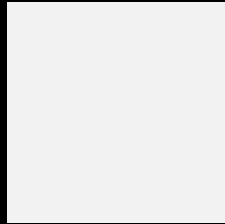
One line in a Dockerfile

Which one makes Redis use more energy?

One run = Redis' own benchmark, from start to finish (about five minutes)



FROM ubuntu



42 MB



FROM alpine



3.8 MB

Image size on Docker Hub, Oct 2026

One line in a Dockerfile

Answer: **Alpine: about as fast, but more energy**

One run = Redis' own benchmark, from start to finish (about five minutes)



Image size on Docker Hub, Oct 2026

Smaller is not always greener.

Case 2 · Debugging an energy regression in Redis

Motivation: the base image changes Redis' energy, and nobody knew why

Server side energy

No battery, and clients do not pay the energy bill, so it gets less attention

Energy tools

Few tools exist to debug energy on servers

Earlier work

base image changes energy use, but why

Redis: a popular database that keeps data in memory, so apps can read it fast

Case 2 · Debugging an energy regression in Redis

Motivation: the base image changes Redis' energy, and nobody knew why

Server side energy

No battery, and clients do not pay the energy bill, so it gets less attention

→ Study a widely used server software: Redis

Energy tools

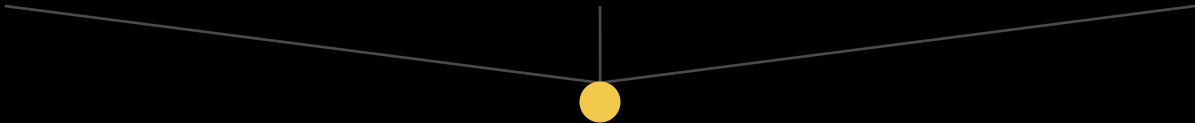
Few tools exist to debug energy on servers

→ Link energy to the code that spends it

Earlier work

base image changes energy use, but why

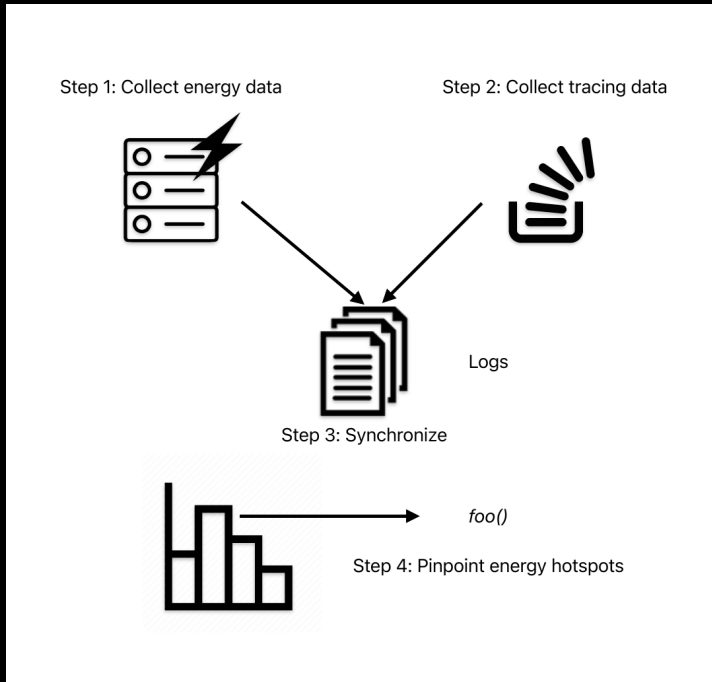
→ Find the root cause



This work: a method to debug energy regressions, applied to Redis

Redis: a popular database that keeps data in memory, so apps can read it fast

Measure, trace, align, then pinpoint the hotspots



Roque et al., ICSE 2025, Fig. 1

RQ1 · Observe + Measure + Localize

Is the Alpine-Ubuntu energy gap real?

RQ2 · Localize

Can the method point to the cause?

RQ3 · Evaluate

Can the cause be isolated?

RQ1 · Is the Alpine-Ubuntu energy gap real?

Answer: **Yes, and the cause is the C library**

Ubuntu with glibc

baseline

Alpine with musl

+8.6% energy

Alpine with glibc

+2.4% energy

Run time is about equal in all three.

RQ2 · Can the method point to the cause?

Same run time, more energy. Where would you look first?

RQ2 · Can the method point to the cause?

Energy run

30×, no tracing



RQ2 · Can the method point to the cause?

Energy run

30×, no tracing



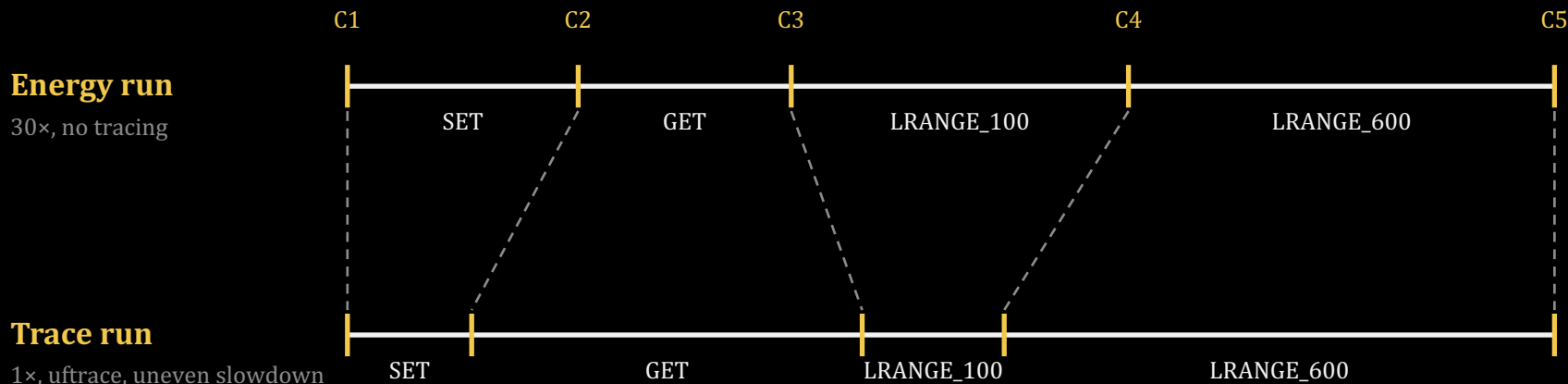
Trace run

1×, ufttrace, uneven slowdown



RQ2 · Can the method point to the cause?

Method: **log checkpoints align the call trace with the power timeline**

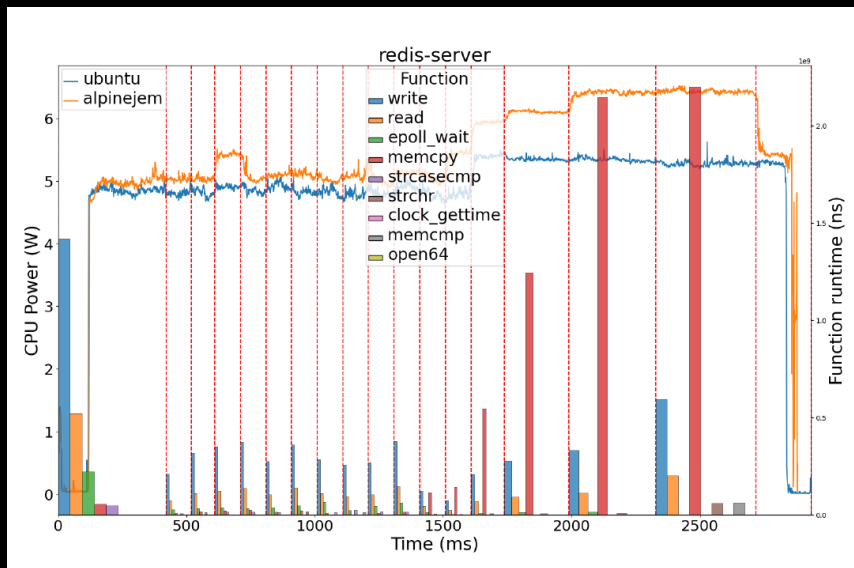


Checkpoints (C1...C5 in this simplified drawing) are unique lines in Redis' log. Between two checkpoints, both runs did the same work, however long it took.

Link the energy to the code that spends it.

RQ2 · Can the method point to the cause?

Answer: **Yes: memcpy dominates where Alpine's power is higher**



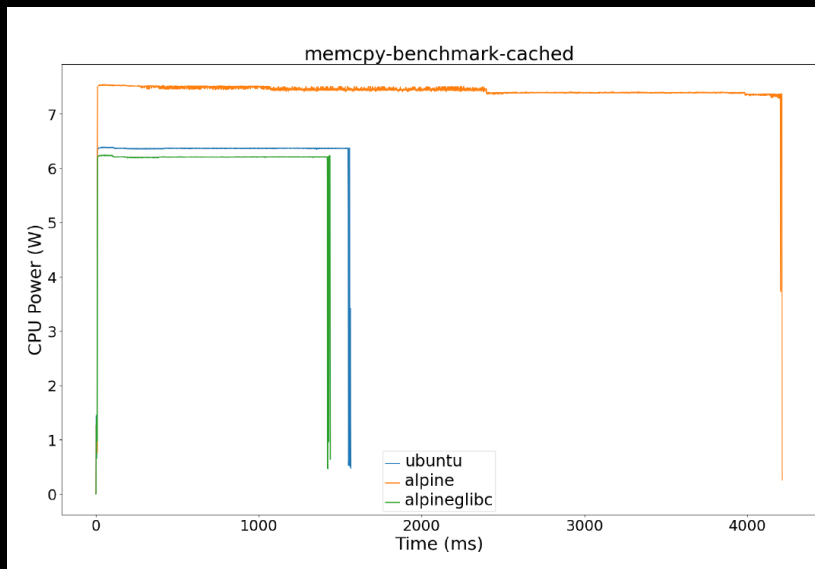
Roque et al., ICSE 2025, Fig. 8 and Table III

Function	Runtime [ns (%)]	# Calls [(%)]
memcpy	6594371865 (39.6)	115270590 (92.6)
write	6100079082 (36.6)	500006 (0.4)
read	2204989695 (13.2)	501557 (0.4)
epoll_wait	701065124 (4.2)	99494 (0.1)
strchr	238221535 (1.4)	2340922 (1.9)
strcasecmp	221697527 (1.3)	1776711 (1.4)
gettimeofday	152283682 (0.9)	1201218 (1.0)
memcmp	150901874 (0.9)	1307754 (1.0)
clock_gettime	117885050 (0.7)	898262 (0.7)
localtime_r	45398704 (0.3)	99495 (0.1)

memcpy: 92.6% of libc calls

RQ3 · Can the cause be isolated?

Answer: **Yes: on Redis-like copies, musl's memcpy uses 3× the energy**



Roque et al., ICSE 2025, Fig. 11 and Table V

Memory → memory: all three equal

Cache → memory (like Redis): **musl 2.6× slower**

Likely why: glibc adds optimized assembly code; musl keeps its code in C.

Software diversity (ILO 6.1): two implementations of one interface, compared.

Case 2: almost all the work is in Localize

Observe	Alpine uses more energy than Ubuntu, in about the same run time
Localize	first the C library, then one function: memcpy
Intervene	glibc put into Alpine: a test of the cause, not a proposed fix
Measure	CPU energy of one core, 30 runs per image
Evaluate	a memcpy-only benchmark confirms it, for Redis-like copies
Trade-offs	not studied; Alpine is popular for its small image

Enrique Barba Roque, Luís Cruz, Thomas Durieux. Unveiling the Energy Vampires: A Methodology for Debugging Software Energy Consumption. ICSE 2025.

You rent a server in the cloud

Can you make it use less energy? How?

Case 3 · Architectural tactics for energy in the cloud

Motivation: energy research served the provider, not the customer

Cloud provider

owns the hardware

can measure its energy

many studies

Cloud customer

rents the machines

gets almost no energy data

hardly any studies

Case 3 · Architectural tactics for energy in the cloud

Motivation: energy research served the provider, not the customer

Cloud provider

owns the hardware

can measure its energy

many studies

Cloud customer

rents the machines

gets almost no energy data

hardly any studies

This work: a catalogue of tactics for cloud customers, and one tactic tested

Three questions, three methods

RQ1	Which tactics exist?	interviews with practitioners
RQ2	What can customers monitor?	a review of the tools
RQ3	What does one tactic save?	one case study

RQ1 · Which tactics can cloud customers use?

Answer: **18 concrete tactics, collected from 17 practitioners**

Three examples

Switch off what you do not use

Add and remove machines as the load changes

Process data close to where it is produced

= edge computing, tested next

Behind all of them is one assumption: fewer resources, less energy.

RQ2 · What can cloud customers monitor?

Answer: **Only coarse estimates, not measurements of their own workload**

Tools in early 2022

Azure, Google

carbon only

AWS

no tool yet

Cloud Carbon Footprint

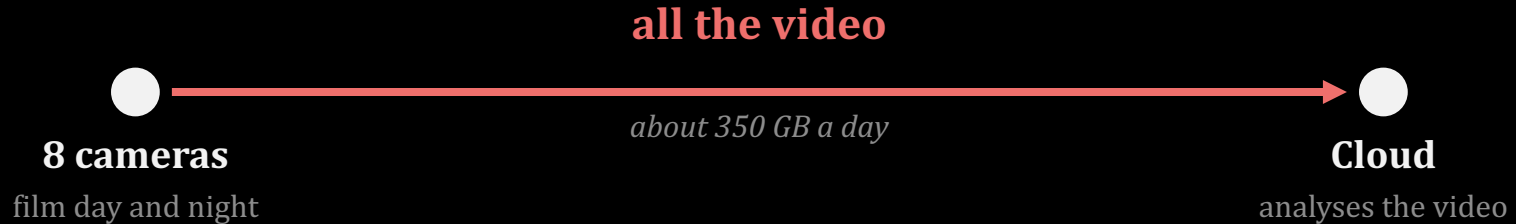
energy, estimated from the bill

open source, not from a provider

RQ3 · Observe: all video goes to the cloud

Observe Localize Intervene Measure Evaluate Trade-offs

Fall detection in elderly care, a real project



The signal: a lot of data leaves the building.

RQ3 · Localize: almost all energy is data transfer

Observe **Localize** Intervene Measure Evaluate Trade-offs

Cloud-only design, energy for one day (an estimate)



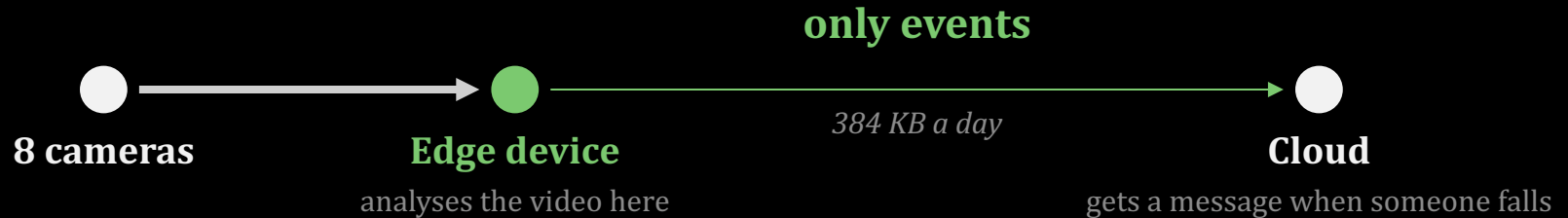
The numbers are from the paper. Putting them side by side is our reading.

Find where the energy goes before you change anything.

RQ3 · Intervene: analyse the video next to the cameras

Observe Localize **Intervene** Measure Evaluate Trade-offs

Edge computing: process the data where it is produced



RQ3 · Measure: both designs, estimated for one day

Observe Localize Intervene **Measure** Evaluate Trade-offs

Cloud only



21.96 kWh

Edge



0.72 kWh

96% less

Estimated from power ratings and published averages. Nothing is measured.

RQ3 · Evaluate: is it really better?

Observe Localize Intervene Measure **Evaluate** Trade-offs

Data transfer

from one average value: 0.06 kWh per GB

Without data transfer

0.72 vs. 1.21 kWh: 40% less

Does it still do its job?

detection quality is not compared

The first two points are our reading of the numbers in the paper.

96% less energy

Would you switch? What could it cost you?

RQ3 · Trade-offs: where it holds, what it costs

Observe Localize Intervene Measure Evaluate **Trade-offs**

Where it holds

only for systems that send a lot of data

Cost

device and staff time: not examined

Reliability

what if the device fails? not examined

Case 3: the case study in six steps

Observe	all the video goes to the cloud, day and night
Localize	almost all the energy is data transfer
Intervene	edge computing: analyse next to the cameras, send only events
Measure	an estimate for one day, not a measurement
Evaluate	96% less; 40% without data transfer; detection quality not compared
Trade-offs	only for systems that send a lot of data; cost and reliability not examined

(The six steps describe the case study)

Sophie Vos, Patricia Lago, Roberto Verdecchia, Ilja Heitlager. Architectural Tactics to Optimize Software for Energy Efficiency in the Public Cloud. ICT4S 2022.

Same six steps, three ways to carry them out

Each case compares alternative implementations: software diversity (ILO 6.1)

	Case 1 · unit test		
Observe	a cost that repeats		
Localize	pick the slowest tests		
Intervene	mocking, mined from developers		
Measure	30 runs, statistics		
Evaluate	mutation score		
Trade-offs	test effectiveness		

Same six steps, three ways to carry them out

Each case compares alternative implementations: software diversity (ILO 6.1)

	Case 1 · unit test	Case 2 · library	
Observe	a cost that repeats	an unexpected contrast	
Localize	pick the slowest tests	align power with a call trace	
Intervene	mocking, mined from developers	a diagnostic swap, no fix	
Measure	30 runs, statistics	30 runs, one CPU core	
Evaluate	mutation score	a memcpy-only benchmark	
Trade-offs	test effectiveness	not studied	

Same six steps, three ways to carry them out

Each case compares alternative implementations: software diversity (ILO 6.1)

	Case 1 · unit test	Case 2 · library	Case 3 · cloud
Observe	a cost that repeats	an unexpected contrast	all video goes to the cloud
Localize	pick the slowest tests	align power with a call trace	split the estimate into parts
Intervene	mocking, mined from developers	a diagnostic swap, no fix	edge computing
Measure	30 runs, statistics	30 runs, one CPU core	an estimate, not a measurement
Evaluate	mutation score	a memcpy-only benchmark	96% less, one key assumption
Trade-offs	test effectiveness	not studied	not studied

Where do intervention ideas come from?

Where do intervention ideas come from?

- Energy bug fix
- Frequency and voltage scaling
- Disabling components
- Using efficient component
- Managing periodic work
- Low power idling
- Timing out
- Lazy initialization
- Improving performance

Moura, Pinto, Ebert, Castor. Mining energy-aware commits. MSR 2015.

Where do intervention ideas come from?

- Energy bug fix
- Frequency and voltage scaling
- Disabling components
- Using efficient component
- Managing periodic work
- Low power idling
- Timing out
- Lazy initialization
- Improving performance

Case 1: improving performance **Case 2:** using efficient component **Case 3:** edge computing fits none exactly.
Say why.

Moura, Pinto, Ebert, Castor. Mining energy-aware commits. MSR 2015.

A map of Green SE research

	Code & libraries	Mobile & web	Testing & CI	AI / ML	Architecture & cloud
Measure <i>how much?</i>	Thread management energy OOPSLA '14 EnergiBridge (tool) arXiv '23	App framework overhead EASE '23	Testing energy in CI AST '24 Mobile testing frameworks TSE '19 • UI test frameworks SAC '26	DL models and training setup SEAA '23 • Serving code SLMs JSS '25	Performance models for energy QEST '23 KPIs for cloud storage ICSA '24
Debug <i>where and why?</i>	Energy Vampires ICSE '25	EDATA for Android MOBILESoft '25 Battery saver issues MOBILESoft '25	From Slow to Green (mining) 2026	—	—
Optimize <i>what helps?</i>	Recommending Java collections EMSE '21 Automatic refactoring JSERD '20	Leafactor refactoring MOBILESoft '17 JS dead code elimination TSE '23	From Slow to Green (mocking) 2026 Batch testing in CI DevOpsSustain '26	Sustainable ML retraining ICT4S '25 Batching for inference SEAA '23 Model selection for ensembles Alware '24	Cloud tactics (edge) ICT4S '22 • Tactics tested in industry ICSA '24 • Sufficient use of the cloud ICT4S '24
Guide <i>what works?</i>	Mining energy-aware commits MSR '15 Energy-aware software manifesto IEEE Softw. '19	Energy patterns, mobile EMSE '19 Energy patterns, web ICSE-SEIS '24	—	Green AI: systematic review WIRES '22 Green tactics for ML systems ICSE-SEIS '24	Cloud tactics catalogue ICT4S '22 Green tactics mined from ROS MSR '21 SAF toolkit ICSA '24

When does a claim hold, and what is still open?

LIMIT OF THE CLAIM

Does it still do the same job?

Case 1: a mocked test can stop catching bugs

Does it hold somewhere else?

Case 2: musl wins or loses depending on the workload and CPU

Was it measured or estimated?

Case 3: 96% comes from a model

Saved here, saved overall?

Sustainability lecture: rebound effects can eat the savings

When does a claim hold, and what is still open?

LIMIT OF THE CLAIM

Does it still do the same job?

Case 1: a mocked test can stop catching bugs

Does it hold somewhere else?

Case 2: musl wins or loses depending on the workload and CPU

Was it measured or estimated?

Case 3: 96% comes from a model

Saved here, saved overall?

Sustainability lecture: rebound effects can eat the savings

OPEN QUESTION

→ What else does a faster test give up? Can a tool tell where mocking is safe?

→ Does the memcpy story hold on Intel, ARM, or other libraries?

→ How can cloud customers get real energy data?

Reading your paper for Week 7

The five required parts of your presentation, with Case 2 as an example

Context

Problem

Solution

Limitations

Gaps

Reading your paper for Week 7

The five required parts of your presentation, with Case 2 as an example

Context	Redis is widely deployed in containers; base images differ
Problem	Alpine uses more energy than Ubuntu, and nobody knows why
Solution	align power with a call trace; cause is memcpy in musl. Meta-solution: using efficient component. Competing: time profilers, which show time, not energy
Limitations	one AMD CPU, CPU energy only, one core; the better library depends on the workload
Gaps	other CPUs and libraries? can the method be automated?

Pick your paper for Week 7

The shared paper list

Write your name next to the paper you pick. Each paper can be picked once.



scan to open the list

Master thesis projects: we are looking for two students

Two MSc thesis projects, in collaboration with ING

- 1 Confidence estimation for ING's RAG-based LLM system
- 2 Stability of ING's RAG-based LLM system

Interested? Email me with your CV: xutong.liu@utwente.nl



What to do right after this lecture

- Review the slides
- Pick your paper for the Week 7 presentations (Friday 16 October), alone or in pairs, not your project team: from the shared paper list (Google Doc)
- Prepare the five parts of your presentation: context, problem, solution, limitations, gaps
- Use today's six steps to read the method: observe, localize, intervene, measure, evaluate, reason about trade-offs
- For Assignment 3: explain your results. Where does the energy go, and what does the difference cost?

Further reading

Read and learn

From Slow to Green: Agentic AI-Powered Mock Refactoring for Python Tests

Liu & Zaidman, preprint 2026 · Case 1

Unveiling the Energy Vampires: A Methodology for Debugging Software Energy Consumption

Barba Roque, Cruz & Durieux, ICSE 2025 · Case 2

Architectural Tactics to Optimize Software for Energy Efficiency in the Public Cloud

Vos, Lago, Verdecchia & Heitlager, ICT4S 2022 · Case 3

Mining Energy-Aware Commits

Moura, Pinto, Ebert & Castor, MSR 2015 · where intervention ideas come from

Estimating the Energy Footprint of Software Systems: a Primer

Castor, arXiv:2407.11611

Building Green Software

Currie, Hsu & Bergman, O'Reilly 2024 · a book for practitioners

Sustainable Software Engineering, TU Delft

an open course: lectures and student projects are online

Organisations

Green Software Foundation

industry non-profit: the SCI standard, a free practitioner course

Green Web Foundation

non-profit: checks if a website runs on green energy; the CO2.js library

Events

ICT4S · GREENS at ICSE

a yearly conference and a workshop; Case 3 was published at ICT4S

Try

EnergiBridge · CodeCarbon

open tools to measure the energy, or estimate the carbon, of your own code

Subscribe

CAT Newsletter

weekly, from ClimateAction.tech: a community of tech workers acting on climate

Sustainability by Numbers

Hannah Ritchie's newsletter: sustainability questions, answered with data